

Fiche Projet Détaillée : USAL59

Projet interdisciplinaire

Institution : Conservatoire national des arts et métiers (Cnam)
Module : USAL59 - Coder une application complexe
Année universitaire : 2025–2026
Modalité : Contrôle continu (rendus du projet, soutenances)
Intervenants : LASHEB Mohamed-Amine & DJEBLI Meihdi

1. Introduction et Objectifs Pédagogiques

Bonjour à toutes et à tous, et bienvenue dans ce projet transversal.

L'objectif de ce module est de développer une application de bout en bout, du recueil des besoins jusqu'au déploiement. Vous allez concevoir une architecture robuste, prototyper des interfaces (UI/UX), développer des fonctionnalités avancées et collaborer efficacement.

2. Sujet : Développement du "LLM Entreprise Hub"

Le Contexte Métier

Notre entreprise utilise de plus en plus l'Intelligence Artificielle. Nos collaborateurs se servent de différents modèles de langage (LLM) pour coder ou analyser des données.

Cependant, nous faisons face à un problème : l'éparpillement des outils et les limites de requêtes des API qui bloquent les équipes. De plus, l'utilisation d'outils externes disparates entraîne une perte totale de l'historique des échanges professionnels.

Votre Mission Centrale

Concevoir et déployer une application centralisée permettant de requêter plusieurs LLM depuis une interface unique, tout en gérant intelligemment les limites de ces modèles et en enregistrant les métriques d'utilisation.

Fonctionnalités clés attendues :

- **Interface unifiée et multi-modèles :**

L'application doit proposer une interface permettant à l'utilisateur de sélectionner et d'utiliser plusieurs modèles de langage (ex : Gemini, Claude, GPT) depuis un même espace.

- L'utilisateur peut choisir dynamiquement le modèle à utiliser pour chaque requête.

- L'application doit intégrer au minimum trois modèles différents.
 - Les modèles sélectionnés doivent être documentés (choix, limites, avantages).
 - Les modèles doivent impérativement proposer une offre gratuite avec des limites d'utilisation.
 - L'objectif est de maximiser l'utilisation des offres gratuites des API, sans recourir à des solutions payantes.
 - L'interface doit afficher et gérer dynamiquement les limites (requêtes, tokens) propres à chaque modèle.
 - **Affichage des quotas en temps réel** : L'interface doit afficher dynamiquement les limites restantes des API (nombre de requêtes et tokens disponibles).
 - **Persistance des données** : Sauvegarde complète des conversations en base de données afin de permettre la reprise des échanges.
 - **Support multimodal** : Gestion de fichiers (PDF, Word, TXT) et d'images pour exploitation par les modèles compatibles.
 - **Accessibilité audio** :
 - **Speech-to-Text** : dictée vocale transformée en texte
 - **Text-to-Speech** : lecture audio des réponses générées
 - **Personnalisation et accessibilité de l'interface** :
 - Mode clair / mode sombre (Dark Mode)
 - Support multilingue (ex : français, anglais)
 - Adaptation de l'interface selon les préférences utilisateur
 - **Moteur de routage et monitoring** : Gestion intelligente des quotas côté back-end avec tableau de bord pour le suivi des usages.
-

3. Le Défi Technique : Gestion des API et des Tokens

Pour concevoir votre système, prenons l'exemple de Google Gemini.

1 token $\approx$ 4 caractères

Le texte envoyé et la réponse générée consomment des tokens.

- **RPM** : 5 requêtes par minute maximum
- **TPM** : 250 000 tokens par minute
- **RPD** : 20 requêtes par jour

Votre défi technique : Interceptor les erreurs HTTP (comme la 429 Too Many Requests), concevoir une logique de routage (attente, bascule de modèle) et renvoyer au front-end les informations de quotas en temps réel.

4. Architecture, Stack Technologique et Exigences

Exemples de technologies

- **Back-end** : Node.js (Express), PHP (Symfony, ApiPlatform), Python (Django), Java, etc.
- **Front-end** : HTML/CSS, JavaScript, React, Vue.js, Angular, Bootstrap, etc.

- **Bases de données** : MySQL, PostgreSQL, MariaDB, NoSQL (MongoDB, Firebase), etc.
- **Outils** : Git, Docker, GitLab CI, Maven, etc.
- **Cloud** : Render, Heroku, Vercel, Scalingo, DigitalOcean, etc.

Exigences techniques incontournables

- Séparation Front-end / Back-end (API REST)
- Modélisation UML avant développement
- Modélisation de la base de données
- Utilisation de Git (obligatoire)
- Déploiement sur le cloud
- Contraintes sur les choix technologiques (par ex : déploiement) :
 - Les solutions techniques doivent privilégier des offres gratuites afin de garantir l'égalité des chances entre les groupes.
 - Chaque choix technologique devra être justifié et documenté.
 - Les solutions exclusivement payantes ne seront pas acceptées.

5. Méthodologie Agile, Sprints et Livrables

Utilisation obligatoire d'un outil Kanban pour organiser et suivre l'avancement du projet.

Un tableau Kanban permet de visualiser les tâches sous forme de colonnes :

- À faire
- En cours
- Terminé

Chaque tâche est représentée par une carte, assignée à un membre de l'équipe et évoluant au fil du projet.

Objectifs :

- Répartir efficacement les tâches entre les membres
- Suivre l'avancement du projet en temps réel
- Identifier les blocages rapidement
- Améliorer la collaboration au sein de l'équipe

Des outils comme Trello ou Notion peuvent être utilisés.

Identité du projet et valorisation professionnelle :

Chaque groupe devra définir une identité propre à son application :

- Un nom de projet
- Un logo
- Une identité visuelle (palette de couleurs, style graphique)

Cette démarche vise à professionnaliser le projet et à permettre sa valorisation dans un portfolio ou sur des plateformes comme LinkedIn.

Phase 1 : Conception (Sprints 1 à 4)

- **Sprint 2 : Le Pitch (Go/No-Go)** Présentation orale courte (5 à 10 minutes) pour valider les choix techniques.

- **Sprint 4 - Livrable 1 (Conception)** Ce livrable constitue un contrat pour la suite du projet :
 - Cahier des charges fonctionnel
 - Design UI/UX (maquettes) : outils recommandés (Figma, Adobe XD, etc.)
 - Diagrammes UML : outils recommandés (draw.io, Lucidchart, etc.)
 - Schéma de base de données : outils recommandés (draw.io, dbdiagram.io, MySQL Workbench, etc.)

Phase 2 : Développement (Sprints 5 à 8)

- **Sprint 8 - Livrable 2 (Réalisation)** Dépôt Git et application déployée en ligne

Phase 3 : Validation et Audit

- **Séance 9 : Soutenance finale (démonstration live)**
 - Audit de conformité par rapport au cahier des charges
-

6. Critères d'évaluation

- Qualité du Code : 35%
 - Conception & Modélisation (UI/UX et UML) : 25%
 - Intégration & Déploiement : 15%
 - Soutenance : 15%
 - Gestion de projet : 10%
-

Conclusion

Prochaine étape : Réunissez-vous, organisez votre équipe et préparez vos questions pour le prochain Sprint.